

1. Student Details

Name:Eklawya Rastogi

Roll Number: 24f3005112

Email: 24f3005112@ds.study.iitm.ac.in

2. Project Details

Project Title: TREKKING MANAGEMENT APP

Problem Statement:

To build a Trekking Management Application web application that allows Admin, Trek Staff, and Users (Trekks) to interact with the system based on their roles.

Approach:

The app was built using Flask as the backend framework with Flask SQL-Alchemy for backend management . It features a role-based access control system and a multi-step approval workflow for Staff and trek approval.

3. AI/LLM Declaration

I used **Gemini** to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency.

The extent of AI/LLM usage is around **15-20%**, limited to mainly for understanding few concepts , guidance and structure of code.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
HTML5	Creating Different pages for each role

5. Database Schema / ER Diagram

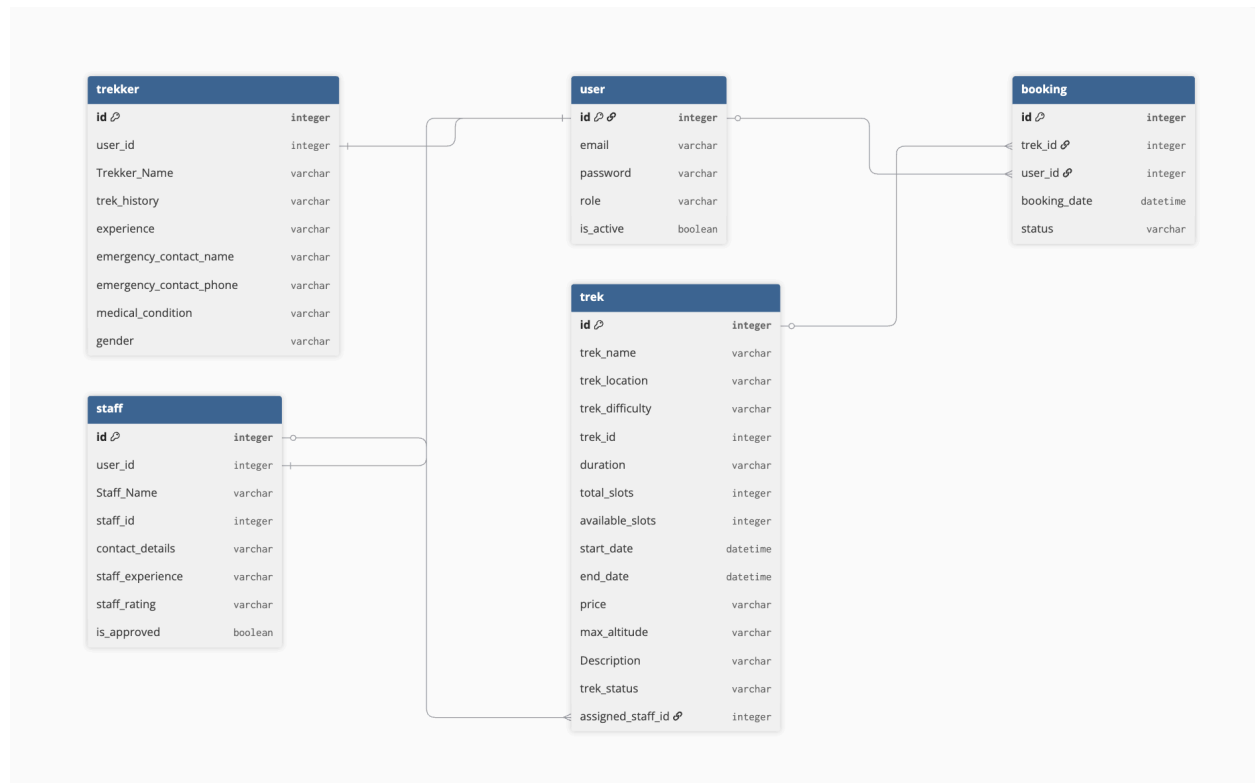
Tables:

- User - Stores id , email , password , role and is_active(status).
- Trek - Stores id , trek name , location , difficulty , trek id , duration , total slots , available slots , start date , end date , price , max altitude , trek description , trek status , assigned staff_id(FK-staff id).
- Staff - id , user_id(FK) , Staff name , staff id(FK) , contact details , experience , rating , is_approved.
- Trekker - id , user_id(FK) , Trekker Name , Trek history , experience , emergency contact name , emergency contact phone , medical condition and gender.
- Bookings - id , trek id(FK) , user_id(FK) , booking date , status(approved , open , close , pending)

Relations-

- **One to One Relation — Between User and Staff/Trekker** Every field guide staff member or trekker is linked to exactly one user account for authentication and access controls via the unique foreign key mapping.

- **One to Many Relation — Between Staff and Trek** A single assigned staff member (field guide) can be associated with or manage multiple trek destination route itineraries.
- **Many to Many Relation — Between User (Trekker) and Trek through Booking** Many users can book a single trek itinerary, and a single user can book or participate across multiple different trek route cycles through the booking ledger table.



6. API Resource Endpoint

Endpoint	Method	Description
<code>/api/login</code>	POST	Authenticate user and generate session

<code>/api/activities</code>	GET	Fetch logged activities for a user
<code>/api/activity/add</code>	POST	Add new activity entry
<code>/api/summary</code>	GET	Return daily/weekly analytics summary

7. Architecture and Features Architecture:

Follows the Model-View-Controller (MVC) pattern where SQLAlchemy handles the Model, Jinja2 handles the View, and Flask routes handle the Controller.

Key Feature: Multi-stage approval logic ensures that staff registrations and open trek itineraries are only accessible after verified verification by the Admin.

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1sBodDDImqw6JH985_iDb9sza1toSQNYV/view?usp=sharing